SYSTEMS ENGINEERING PORTFOLIO · LINUX / FOSS

50 programs, and something outside each one that decides if it is right.

Every project here is built against an **external oracle** — gcc and clang, the Linux kernel, `dash`, GNU tar, readelf, SQLite, Python's own regex engine. Not one of them is the judge of its own correctness. That constraint is the whole method, and it is why **10 of the 18 defects recorded here were in the test harness rather than in the code**.

paperpaladin
github.com/paperpaladin

50	1314	63,916	80,204	50/50
REPOSITORIES	COMMITS	LINES OF SOURCE	LINES OF TESTS	SUITES GREEN

Every figure in this document is generated from the repositories themselves: commit counts from `git`, line counts from the files, test counts from running each suite. Nothing is transcribed by hand.

The method

A test you wrote, checking code you wrote, against a number you worked out yourself, proves that you were consistent. It does not prove you were right. **Every project here is measured against an authority it does not control.**



How these 50 compose. Each arrow is a dependency recorded in a PLAN.md. The three process tools in the bottom lane act on all the others: **unrepro** found a real reproducibility defect in eight of the 50, **commitq** gates every commit in all of them, and **srcdiff** audits published artifacts against the repositories they claim to come from.

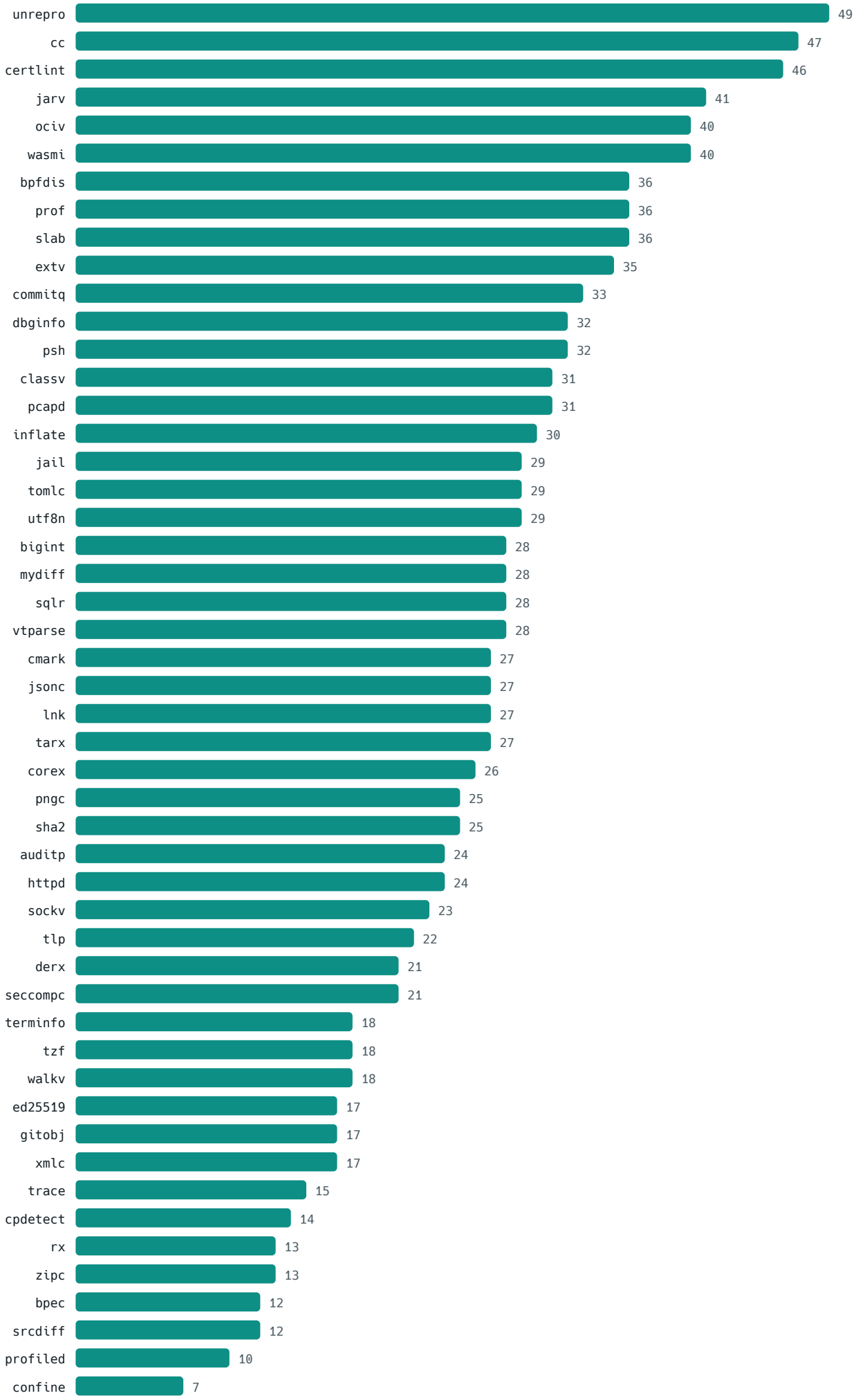
WHY THIS MATTERS MORE THAN IT SOUNDS

The first run of almost every harness in this project passed. The useful question is never “do my tests pass” but **“have I watched this harness fail?”** Where that question got asked, it usually turned out the answer was no — and the suite had been reporting success because it was silently doing nothing.



Defects recorded in commit messages and READMEs across all every repository here. Counted once per root cause.

Scale

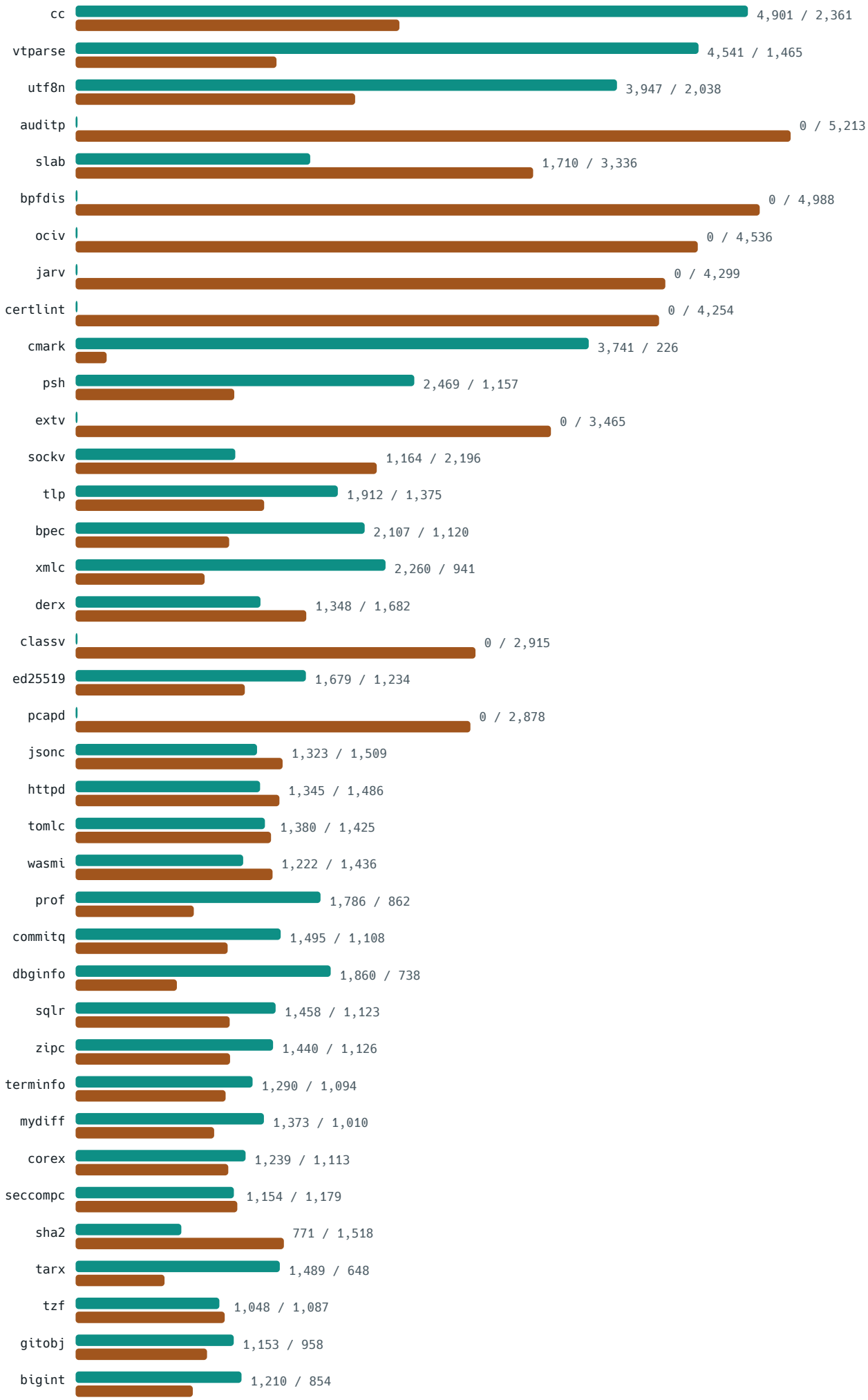

unrepro 49
cc 47
certlint 46
jarv 41
ociv 40
wasmi 40
bpfdis 36
prof 36
slab 36
extv 35
commitq 33
dbginfo 32
psh 32
classv 31
pcapd 31
inflate 30
jail 29
tomlc 29
utf8n 29
bigint 28
mydiff 28
sqlr 28
vtparse 28
cmark 27
jsonc 27
lnk 27
tarx 27
corex 26
pngc 25
sha2 25
auditp 24
httpd 24
sockv 23
tlp 22
derx 21
seccompc 21
terminfo 18
tzf 18
walkv 18
ed25519 17
gitobj 17
xmlc 17
trace 15
cpdetect 14
rx 13
zipc 13
bpec 12
srcdiff 12
profiled 10
confine 7

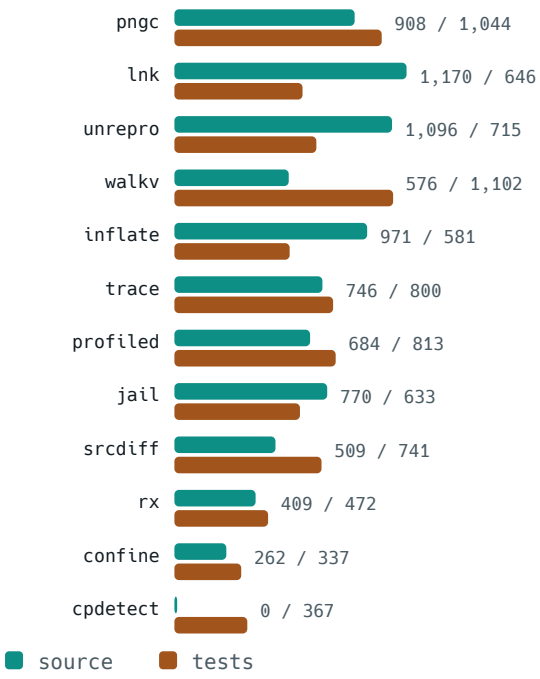
Commits per repository. Granularity is deliberate: every commit builds, verified by removing build artifacts and rebuilding each one in turn.

Source against tests

80,204 lines of tests to 63,916 of source. The ratio is the point: in several of these the harness **is** the project, and the program under test is

scaffolding for it.


cc
4,901 / 2,361
vtparse
4,541 / 1,465
utf8n
3,947 / 2,038
auditp
0 / 5,213
slab
1,710 / 3,336
bpfdis
0 / 4,988
ociv
0 / 4,536
jarv
0 / 4,299
certlint
0 / 4,254
cmark
3,741 / 226
psh
2,469 / 1,157
extv
0 / 3,465
sockv
1,164 / 2,196
tlp
1,912 / 1,375
bpec
2,107 / 1,120
xmlc
2,260 / 941
derx
1,348 / 1,682
classv
0 / 2,915
ed25519
1,679 / 1,234
pcapd
0 / 2,878
jsonc
1,323 / 1,509
httpd
1,345 / 1,486
tomlc
1,380 / 1,425
wasmi
1,222 / 1,436
prof
1,786 / 862
commitq
1,495 / 1,108
dbginfo
1,860 / 738
sqlr
1,458 / 1,123
zipc
1,440 / 1,126
terminfo
1,290 / 1,094
mydiff
1,373 / 1,010
corex
1,239 / 1,113
seccompc
1,154 / 1,179
sha2
771 / 1,518
tarx
1,489 / 648
tzf
1,048 / 1,087
gitobj
1,153 / 958
bigint
1,210 / 854



Lines of source and lines of test per repository, sorted by total. Labels read source / tests.

A NUMBER WORTH DISTRUSTING

Test *counts* are not comparable between these repos and are deliberately not charted together. `wasmi`'s 11,833 are generated arithmetic cases; `jail`'s 101 are hand-written assertions against kernel behaviour, and each one took longer to get right than a thousand of the former. Plotting them on one axis would imply a comparison that does not exist.

REPO	WHAT THE SUITE ACTUALLY REPORTS	PLAN
`auditp`	92 passed, 0 failed	27/34
`bigint`	107573 agree, 0 disagree	21/28
`bpec`	46 passed, 0 failed	11/11
`bpfdis`	105 passed, 0 failed	34/36
`cc`	434 passed, 0 failed (2 suites)	28/33
`certlint`	24 passed, 0 failed	54/57
`classv`	106469 passed, 0 failed	36/38
`cmark`	22 passed, 0 failed	19/19
`commitq`	173 passed, 0 failed	8/17
`confine`	25 passed, 0 failed	4/12
`corex`	80 passed, 0 failed, 0 skipped	18/22
`cpdetect`	270 passed, 0 failed	22/31
`dbginfo`	118 passed, 0 failed	19/29
`derx`	218 passed, 0 failed, 0 skipped	12/22

REPO	WHAT THE SUITE ACTUALLY REPORTS	PLAN
ed25519	43 passed, 0 failed	10/12
extv	692 passed, 0 failed	45/49
gitobj	2365 passed, 0 failed	7/24
httpd	258 passed, 0 failed, 0 skipped	19/28
inflate	169 passed, 0 failed	22/30
jail	101 passed, 0 failed, 0 skipped	15/23
jarv	14996 passed, 0 failed	56/56
jsonc	386 passed, 0 failed	18/22
lnk	49 passed, 0 failed	19/31
mydiff	2104 passed, 0 failed	15/18
ociv	574 passed, 0 failed	44/46
pcapd	219 passed, 0 failed	48/51
pngc	74 passed, 0 failed	18/24
prof	29 passed, 0 failed, 0 skipped	21/30
profiled	60 passed, 0 failed	12/12
psh	251 agree, 0 disagree, 0 skipped	21/30
rx	3020 agree, 0 disagree, 0 skipped (seed=0)	6/16
seccompc	130 passed, 0 failed, 1 skipped	11/18
sha2	251 passed, 0 failed	18/23
slab	76 passed, 0 failed, 0 skipped	28/30
sockv	139 passed, 0 failed, 1 skipped	12/21
sqlr	171 agree, 0 disagree	18/28
srcdiff	56 passed, 0 failed	7/17
tarx	172 passed, 0 failed	8/19
terminfo	40 passed, 0 failed	8/8
tlp	130 passed, 0 failed	13/17
tomlc	634 passed, 0 failed	18/23
trace	62 passed, 0 failed	12/20
tzf	35 passed, 0 failed	7/13
unrepro	121 passed, 0 failed	13/24
utf8n	118 passed, 0 failed	18/21
vtparse	118 passed, 0 failed	11/20

REPO	WHAT THE SUITE ACTUALLY REPORTS	PLAN
`walkv`	704 scenarios checked, 0 failed	12/19
`wasmi`	11910 passed, 0 failed	26/35
`xmlc`	219 passed, 0 failed	10/18
`zipc`	120 passed, 0 failed	8/17

The repositories

Grouped by what kind of thing they have to survive. Each card names the authority that decides whether it is correct, and the one property

that, if violated, means the project failed.

Toolchain

Programs that build or run other programs.

cc C compiler for x86-64

47 commits · 4,901 lines · 2,361 test lines

cc → gcc AND clang must agree on every case

HARD INVARIANT Self-host fixpoint: cmp cc2 cc3

RESULT 434 passed, 0 failed (2 suites)

PLAN 28 done, 5 queued

wasmi WebAssembly interpreter

40 commits · 1,222 lines · 1,436 test lines

wasmi → Expectations computed independently in Python

HARD INVARIANT A malformed module is rejected or traps, never corrupts the host

RESULT 11910 passed, 0 failed

PLAN 26 done, 9 queued

bpfdis eBPF object disassembler and static verifier model

36 commits · 0 lines · 4,988 test lines

bpfdis → bpftool and llvm-objdump, instruction by instruction, with the kernel ve

HARD INVARIANT Every instruction agrees with both disassemblers, and the corpus is held to a floor of distinct constructs

RESULT 105 passed, 0 failed

PLAN 34 done, 2 queued

psh POSIX shell subset

32 commits · 2,469 lines · 1,157 test lines

psh → dash: stdout AND exit status

HARD INVARIANT Identical stdout and status to dash on every corpus script

RESULT 251 agree, 0 disagree, 0 skipped

PLAN 21 done, 9 queued

classv JVM class file reader and bytecode verifier model

31 commits · 0 lines · 2,915 test lines

classv → the JVM's own verifier on accept and reject, and javap on the disassemb

HARD INVARIANT: Agreement with the real verifier is on which instruction was rejected, not only on the verdict

RESULT: 106469 passed, 0 failed

PLAN: 36 done, 2 queued

mydiff Myers diff and patch

28 commits · 1,373 lines · 1,010 test lines

mydiff → GNU diff byte-identical, and GNU patch applies our output

HARD INVARIANT: The loop closes both ways: our diff feeds GNU patch, GNU's diff feeds our -p

RESULT: 2104 passed, 0 failed

PLAN: 15 done, 3 queued

lnk Static ELF linker

27 commits · 1,170 lines · 646 test lines

lnk → The kernel runs the binary, or it does not

HARD INVARIANT: Every executable emitted runs, or we refused to emit it

RESULT: 49 passed, 0 failed

PLAN: 19 done, 12 queued

gitobj git object database reader and writer

17 commits · 1,153 lines · 958 test lines

gitobj → The object name itself: git chose the filename, we recompute the SHA-1.

HARD INVARIANT: Every object read hashes back to its own name, loose or packed, through the whole delta chain

RESULT: 2365 passed, 0 failed

PLAN: 7 done, 17 queued

Formats & parsers

Programs that read a specification exactly.

dbginfo — DWARF debug-info parser

32 commits · 1,860 lines · 738 test lines

dbginfo → readelf --debug-dump

HARD INVARIANT	Every fact printed agrees with readelf
RESULT	118 passed, 0 failed
PLAN	19 done, 10 queued

inflate — DEFLATE / gzip codec

30 commits · 971 lines · 581 test lines

inflate → gzip -d accepts our output, both directions

HARD INVARIANT	gzip accepts all we emit; we decode all it emits
RESULT	169 passed, 0 failed
PLAN	22 done, 8 queued

tomlc — TOML 1.0.0 parser

29 commits · 1,380 lines · 1,425 test lines

tomlc → toml-test, plus Python's stdlib tomllib

HARD INVARIANT	Valid documents parse to the expected value, not merely without error
RESULT	634 passed, 0 failed
PLAN	18 done, 5 queued

utf8n — Unicode normalisation, all four forms

29 commits · 3,947 lines · 2,038 test lines

utf8n → NormalizationTest.txt from the UCD: 806,840 assertions

HARD INVARIANT	Normalising twice equals normalising once, for every form on every input
RESULT	118 passed, 0 failed
PLAN	18 done, 3 queued

sqlr — SQLite file-format reader

28 commits · 1,458 lines · 1,123 test lines

sqlr → libsqlite3 itself

HARD INVARIANT	Byte-identical output to SQLite; malformed input errors, never crashes
RESULT	171 agree, 0 disagree
PLAN	18 done, 10 queued

vtparse VT500 parser and wcwidth

28 commits · 4,541 lines · 1,465 test lines

vtparse → The published DEC state table, all 3584 pairs

HARD INVARIANT	Never allocates, never reads past its byte, matches the table
RESULT	`118 passed, 0 failed`
PLAN	11 done, 9 queued

cmark CommonMark parser and HTML renderer

27 commits · 3,741 lines · 226 test lines

cmark → The specification's own spec.json: 652 exact markdown/HTML pairs

HARD INVARIANT	Output is byte-identical to the spec's expected HTML, nothing normalised
RESULT	`22 passed, 0 failed`
PLAN	19 done, 0 queued

jsonc Strict RFC 8259 JSON parser

27 commits · 1,323 lines · 1,509 test lines

jsonc → JSONTestSuite: 95 must-accept, 188 must-reject, 35 pinned choices

HARD INVARIANT	A crash can never score as a rejection; exit status outside {0,1} is a failure
RESULT	`386 passed, 0 failed`
PLAN	18 done, 4 queued

tarx tar reader and writer

27 commits · 1,489 lines · 648 test lines

tarx → GNU tar, bidirectional interop

HARD INVARIANT	GNU tar agrees both ways, and no member escapes the destination
RESULT	`172 passed, 0 failed`
PLAN	8 done, 11 queued

pngc PNG decoder, all colour types and Adam7

25 commits · 908 lines · 1,044 test lines

pngc → PngSuite: 162 images that must decode, 14 that must be refused

HARD INVARIANT	A corrupt file is refused by CRC, never rendered as a wrong image
RESULT	`74 passed, 0 failed`
PLAN	18 done, 6 queued

sha2 SHA-2 family, HMAC and PBKDF2

25 commits · 771 lines · 1,518 test lines

sha2 → NIST CAVP vectors, plus hashlib, coreutils and openssl agreeing

HARD INVARIANT: Byte-exact against published vectors; not constant-time, and says so first

RESULT: `251 passed, 0 failed`

PLAN: 18 done, 5 queued

tlp metamorphic SQL logic-bug finder

22 commits · 1,912 lines · 1,375 test lines

tlp → A relation SQLite must satisfy with itself, so no second engine is needed;

HARD INVARIANT: Every disagreement reported is a real one, and a run that finds nothing is distinguishable from a run that tested nothing

RESULT: `130 passed, 0 failed`

PLAN: 13 done, 4 queued

derx DER reader and X.509 printer

21 commits · 1,348 lines · 1,682 test lines

derx → openssl asn1parse and x509, as a diff

HARD INVARIANT: Every element matches asn1parse byte for byte, or is refused with a message

RESULT: `218 passed, 0 failed, 0 skipped`

PLAN: 12 done, 10 queued

terminfo compiled-terminfo reader

18 commits · 1,290 lines · 1,094 test lines

terminfo → ncurses as a subprocess: infocmp for every capability of all 44 terminals

HARD INVARIANT: Every capability reported is the one infocmp reports, because a positional format can be parsed perfectly and named wrongly

RESULT: `40 passed, 0 failed`

PLAN: 8 done, 0 queued

tzf TZif timezone file reader

18 commits · 1,048 lines · 1,087 test lines

tzf → zdump and Python's zoneinfo must both agree

HARD INVARIANT: Every instant answered agrees with both readers; malformed input is refused

RESULT: `35 passed, 0 failed`

PLAN: 7 done, 6 queued

ed25519 Ed25519 and X25519, built up from the field

17 commits · 1,679 lines · 1,234 test lines

ed25519 → A second Ed25519 in Python written from the RFC algorithm rather than

HARD INVARIANT: Every layer is differenced against an implementation sharing no code with it, because sign-then-verify is not evidence

RESULT: `43 passed, 0 failed`

PLAN: 10 done, 2 queued

xmlc XML 1.0 well-formedness checker and canonicaliser

17 commits · 2,260 lines · 941 test lines

xmlc → The W3C XML Conformance Test Suite: 2585 catalogued cases, vendored

HARD INVARIANT: Acceptance is exactly the XML 1.0 Fifth Edition grammar, and no input crashes, overreads or leaks

RESULT: `219 passed, 0 failed`

PLAN: 10 done, 8 queued

zipc ZIP archive reader and writer

13 commits · 1,440 lines · 1,126 test lines

zipc → Info-ZIP unzip 6.00 and zip 3.0, bidirectionally, across every method and

HARD INVARIANT: The central directory is authoritative, and no member can ever be written outside the destination

RESULT: `120 passed, 0 failed`

PLAN: 8 done, 9 queued

bpec Byte-level BPE, exact to the reference

12 commits · 2,107 lines · 1,120 test lines

bpec → The HuggingFace tokenizers library, token for token

HARD INVARIANT: The same token ids as the reference, in the same order

RESULT: `46 passed, 0 failed`

PLAN: 11 done, 0 queued

Systems

Programs that must survive the machine misbehaving.

slab LD_PRELOAD malloc replacement

36 commits · 1,710 lines · 3,336 test lines

slab → Real programs must produce identical output

HARD INVARIANT	Two live allocations never overlap; every pointer aligned
RESULT	`76 passed, 0 failed, 0 skipped`
PLAN	28 done, 2 queued

extv ext2, ext3 and ext4 reader and consistency checker

35 commits · 0 lines · 3,465 test lines

extv → dumpe2fs and debugfs field by field, and e2fsck in both directions

HARD INVARIANT	Clean images are called clean and corrupt ones refused, across block sizes and feature flags that are counted
RESULT	`692 passed, 0 failed`
PLAN	45 done, 4 queued

jail Landlock sandbox

29 commits · 770 lines · 633 test lines

jail → The kernel: denied means EACCES, verified

HARD INVARIANT	An ungranted path is never reachable; the child cannot relax itself
RESULT	`101 passed, 0 failed, 0 skipped`
PLAN	15 done, 8 queued

bigint Arbitrary-precision integers

28 commits · 1,210 lines · 854 test lines

bigint → Python's native int, which is itself arbitrary precision

HARD INVARIANT	Every operation matches Python exactly, including floor division of negatives
RESULT	`107573 agree, 0 disagree`
PLAN	21 done, 7 queued

httpd HTTP/1.1 static file server

24 commits · 1,345 lines · 1,486 test lines

httpd → curl as reference client, RFC 9112 for the error cases

HARD INVARIANT No request can read a byte outside the document root

RESULT `258 passed, 0 failed, 0 skipped`

PLAN 19 done, 9 queued

sockv netlink socket enumerator

23 commits · 1,164 lines · 2,196 test lines

sockv → ss from iproute2, both directions

HARD INVARIANT Every socket the kernel reports is listed once with every field as given

RESULT `139 passed, 0 failed, 1 skipped`

PLAN 12 done, 9 queued

seccompc seccomp-BPF policy compiler

21 commits · 1,154 lines · 1,179 test lines

seccompc → The kernel, plus an i386 probe driving the 32-bit bypass

HARD INVARIANT A syscall the policy did not permit never runs, by any entry point

RESULT `130 passed, 0 failed, 1 skipped`

PLAN 11 done, 7 queued

walkv Write-ahead-log key-value store

18 commits · 576 lines · 1,102 test lines

walkv → A crash injected at every I/O boundary

HARD INVARIANT Recovered state is a prefix with no holes

RESULT `704 scenarios checked, 0 failed`

PLAN 12 done, 7 queued

rx Linear-time regex engine

13 commits · 409 lines · 472 test lines

rx → Python's re module

HARD INVARIANT A pc enters a thread list at most once per input position

RESULT `3020 agree, 0 disagree, 0 skipped (seed=0)`

PLAN 6 done, 10 queued

Observability

Programs that watch other programs without disturbing them.

prof Sampling CPU profiler

36 commits · 1,786 lines · 862 test lines

prof → A planted hot loop with known 10:1 ratios

HARD INVARIANT Profiling does not change the profiled program

RESULT 29 passed, 0 failed, 0 skipped

PLAN 21 done, 9 queued

pcapd pcapng reader and TCP stream reassembler

31 commits · 0 lines · 2,878 test lines

pcapd → tshark per field and per reassembled byte, with tcpdump as a second de

HARD INVARIANT Streams match tshark byte for byte, and the overlapping-segment policy is stated rather than silently chosen

RESULT 219 passed, 0 failed

PLAN 48 done, 3 queued

corex ELF core dump reader

26 commits · 1,239 lines · 1,113 test lines

corex → gdb: registers, backtrace and mappings compared value by value

HARD INVARIANT Every value printed agrees with gdb, or it is not printed

RESULT 80 passed, 0 failed, 0 skipped

PLAN 18 done, 4 queued

auditp Linux kernel audit log parser and event correlator

24 commits · 0 lines · 5,213 test lines

auditp → ausearch and aureport, compared field by field, with syscall numbers cr

HARD INVARIANT Every field agrees with ausearch or is counted as a refusal; the JSON form carries bytes no encoding can represent

RESULT 92 passed, 0 failed

PLAN 27 done, 7 queued

trace ptrace syscall tracer

15 commits · 746 lines · 800 test lines

trace → strace -c and strace -f -c

HARD INVARIANT Tracing does not change the traced program

RESULT 62 passed, 0 failed

PLAN 12 done, 8 queued

cpdetect Exact change point detection for telemetry

14 commits · 0 lines · 367 test lines

cpdetect → brute force enumeration, which is the answer rather than a second opir

HARD INVARIANT The dynamic programme equals the enumerated minimum, PELT equals unpruned optimal partitioning, and recovery is reported beside false positives

RESULT 270 passed, 0 failed

PLAN 22 done, 9 queued

profiled Derive a seccomp policy by watching a program run

10 commits · 684 lines · 813 test lines

profiled → The kernel: a denied syscall kills the process, read from wait4

HARD INVARIANT A syscall absent from the profile is killed, and the kernel says so

RESULT 60 passed, 0 failed

PLAN 12 done, 0 queued

confine What a process can actually do

7 commits · 262 lines · 337 test lines

confine → A process built with unshare and setpriv, reported back exactly

HARD INVARIANT A line says measured only if the operation was attempted this run

RESULT 25 passed, 0 failed

PLAN 4 done, 8 queued

Supply chain & process

Programs that check the other programs.

unrepro — Build-reproducibility differ

49 commits · 1,096 lines · 715 test lines

unrepro → Planted causes, each asserting its diagnosis

HARD INVARIANT: Never report clean on a build that is not reproducible

RESULT: 121 passed, 0 failed

PLAN: 13 done, 11 queued

certlint — X.509 chain validator and CA/Browser Forum linter

46 commits · 0 lines · 4,254 test lines

certlint → OpenSSL and GnuTLS, which disagree about seven certificates, each dis

HARD INVARIANT: Every parsed field and chain decision matches at least one reference, and refusals are counted apart from disagreements

RESULT: 24 passed, 0 failed

PLAN: 54 done, 3 queued

jarv — JAR signature verifier written from scratch

41 commits · 0 lines · 4,299 test lines

jarv → jarsigner, on acceptance and on rejection, with openssl reading the PKC

HARD INVARIANT: Every entry is reported as signed, unsigned or tampered, and an entry absent from the manifest is never silently covered

RESULT: 14996 passed, 0 failed

PLAN: 56 done, 0 queued

ociv — OCI image verifier and layer analyser

40 commits · 0 lines · 4,536 test lines

ociv → skopeo for manifests and digests, GNU tar for what extraction actually d

HARD INVARIANT: Every digest is checked, and a file a later layer deleted is still reported in the layer that carries it

RESULT: 574 passed, 0 failed

PLAN: 44 done, 2 queued

commitq — Multi-repo commit gate and GUI

33 commits · 1,495 lines · 1,108 test lines

commitq → Throwaway repos in a temp directory

HARD INVARIANT: No commit is ever created with a date other than now

RESULT: 173 passed, 0 failed

PLAN: 8 done, 9 queued

srcdiff — Supply-chain artifact auditor

12 commits · 509 lines · 741 test lines

srcdiff → Real xz upstream artifacts

HARD INVARIANT: No false negatives in HIGH, few enough FPs to stay visible

RESULT: 56 passed, 0 failed

PLAN: 7 done, 10 queued

What went wrong in the harness

These are the defects worth reading. In each case the code looked fine, the suite was green, and **the thing that was broken was the thing meant to find breakage.**

`inflate` **`make test-asan` could not fail**

Reject tests assert exit code exactly 1; AddressSanitizer also exits 1. Found by deleting the classic `dist > o->n` bounds check and watching 146 tests pass while ASan printed a heap overflow to stderr. Fixing the exit code revealed that every malformed input had also been leaking 192KB, invisible for the same reason.

`walkv` **Half the crash matrix never crashed**

In torn-write mode the `fsync` boundary called the injection hook and discarded its return value, so every fsync scenario ran to completion and reported a perfect result.

`walkv` **Four planted defects, only one caught**

Mutation testing showed the matrix crashed, recovered, verified and then *stopped* — nothing ever wrote after a recovery, so a subtly wrong recovery was unobservable. Adding that one scenario took the suite from 83 to 163 and the defect now fails 20 of them.

`tarx` **Every symlink assertion was vacuous**

`diff -r` dereferences symbolic links, so the comparisons silently compared targets instead of links, and a dangling link aborted the diff outright.

`srcdiff` **Sixteen false positives on a clean release**

Against the genuine xz-5.4.5 tarball and its git tag, every HIGH finding was wrong — and each was an `m4/*.m4` file absent from git, the exact shape of the attack it exists to find. The real signal would have been one line in seventeen.

`dbginfo` **ASan's exit code collided with “malformed input”**

Both are 1, so the robustness sweeps would have scored a genuine heap overflow as a clean diagnostic.

`unrepro` **Reported REPRODUCIBLE because `make` did nothing**

Pointed at four sibling repos it compared zero outputs and called them clean. Zero outputs is now *inconclusive*.

`commitq` **The commit gate was not gating 12 of 18 repos**

The test command was a hardcoded table of six names; everything else returned success, so two thirds of the project could be committed without running anything.

`slab` **gcc deleted the oracle**

At `-O2` gcc knows what `calloc` promises, so it proved the zero-checking loop could not observe a non-zero byte and removed it. An allocator that never zeroed anything passed cleanly.

`cc` **Five hand-computed expectations were wrong**

Each cost a detour through “is this a codegen bug?” The compiler was right every time. `t/addcase.sh` now takes the expected value from gcc and refuses the case if clang disagrees.

What went wrong in the code

Found by the oracles, never by re-reading the source. Each of these produced plausible output and passed every check that existed at the time.

`rx` **The linear-time guarantee was quietly broken**

Injecting a start thread at every input position collided with the generation marks — and that dedup *is* the guarantee. The one bug that destroyed the project's only interesting property.

`lnk` **A binary the kernel refused to run**

File offset and virtual address were aligned independently. `BASE 0x400000` is divisible by every power of two up to 4MB, so they stayed in step by arithmetic luck until an 8MB-aligned array broke the congruence. The fix made the whole class unemittable rather than merely fixed.

`jail` **A perfect sandbox applied to the wrong process**

The rule-installing loop reused the argument parser's index, so `jail` built a flawless ruleset and then exec'd `--ro`. Every internal invariant held; only a real program reporting a real errno could see it.

`dbginfo` **Strings resolved to different real strings**

clang emits `DW_AT_name` as `strx1` before declaring `str_offsets_base` on the same DIE. Every lookup succeeded, no error, no marker — the second unit was simply named `main.c` where readelf said `lib.c`.

`trace` **A SIGTERM death reported as exit 0**

After passing a signal through, the tracee was already running, so the next `PTRACE_SYSCALL` hit `ESRCH`. Reintroduced once by a later refactor and caught again immediately by the same test.

`prof` **A race against the dynamic loader**

A one-shot `/proc/PID/maps` snapshot raced `ld.so`, truncating libc frames non-deterministically.

`sqlr` **A closing paren ended the column list**

Depth was decremented and tested in the same iteration, so the `)` of `VARCHAR(10,2)` looked like the end of the CREATE statement. `SELECT *` returned one column of four, with no error.

`srcdiff` **A funding link parsed as a repository**

`github.com/sponsors/hynek` resolved to owner *sponsors*. No offline test could reach it: the bug was in metadata resolution, and the tests all covered the comparison.

HOW THIS IS KEPT HONEST

Every commit builds. Verified by checking out each one in turn, deleting build artifacts so a stale binary cannot fake a pass, and rebuilding.

All 50 build reproducibly. Proven by `unrepro`, one of them, which found that eight of its siblings were not — `-g` writes the compiler's working directory into DWARF `.debug_line_str`. Every repo compiling with `-g` was affected and every repo without it was already clean, which is independent confirmation of the cause rather than a plausible guess.

No commit can be gated by a suite that does not run. `commitq` runs each repo's tests against the working tree before committing and refuses when they fail — or when a repo has no discoverable suite at all, which was a real bug it had for two thirds of the project.

167 places take a deliberately simple route, each marked `shortcut:` in the source and naming both the ceiling it accepts and the upgrade path. 967 planned steps are done and 315 remain, ordered, each recording why it is next.

Generated from the repositories by `.portfolio/build.py`. Data-fill colours validated against the paper surface with the dataviz palette checker: worst CVD ΔE 12.6, normal-vision ΔE 21.0, all six checks pass.